

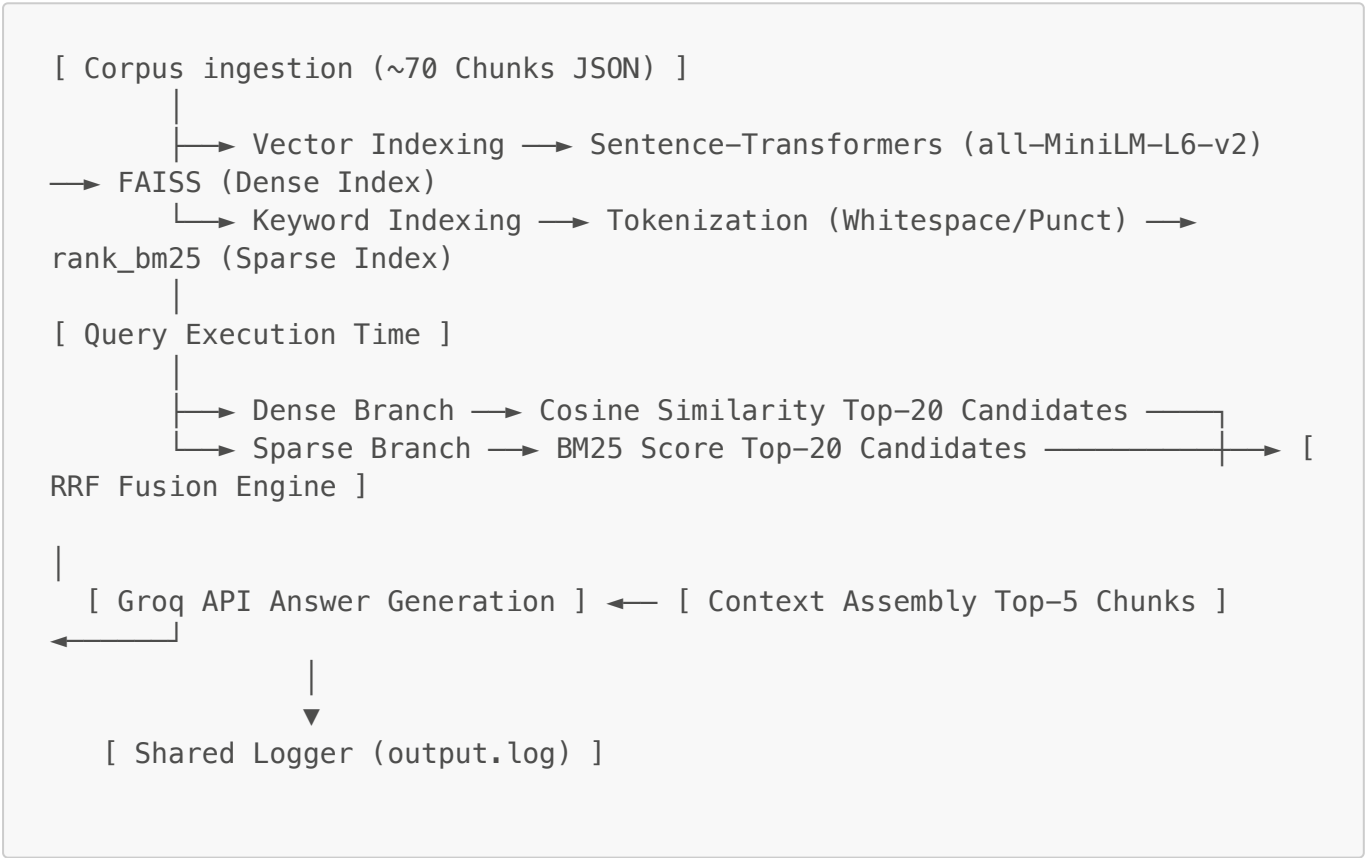
guide.md

SECTION 1: Project Architecture & Overview

End-to-End Pipeline Topology

This project implements an industry-standard, production-grade Advanced Retrieval-Augmented Generation (RAG) architecture using a hybrid retrieval mechanism. Hybrid retrieval addresses a core vulnerability of modern vector-only search engines: the inability to consistently match highly specific, keyword-exact identifiers like policy numbers, clause sub-indices, legal sections, or Harmonized System (HS) tariff codes, while maintaining semantic understanding across general prose.

The end-to-end data and execution flow operates along the following pipeline:



1. **Ingestion & Dual Indexing:** A synthetic heterogeneous corpus representing corporate insurance policies, legal statutes, tax regulations, and customs tariff documents is read from disk. This corpus undergoes two separate indexing paths:
- **Dense Vector Indexing:** Text chunks are passed through a local transformer model to generate dense semantic embeddings, which are loaded into an in-memory matrix index configured for cosine similarity tracking.
 - **Sparse Keyword Indexing:** Text chunks are tokenized into explicit term-frequency representations and indexed using a probabilistic relevance model that weights terms based on their document frequency across the collection.

2. **Query-Time Dual Retrieval:** When a query enters the engine, it is concurrently dispatched to both indexes. The dense retriever returns its top 20 most semantically relevant chunks based on vector proximity. The sparse retriever returns its top 20 most linguistically relevant chunks based on exact term matches.
3. **Reciprocal Rank Fusion (RRF):** The two independent candidate lists (each containing up to 20 items) are fed into the fusion engine. RRF scores each chunk by computing the sum of the reciprocals of its ranks in both lists. By utilizing a constant factor ($k = 60$), the algorithm ensures that outliers or single-list high rankings do not disproportionately skew the final distribution, completely bypassing the need to normalize raw scores across radically incompatible math domains (cosine distances vs. unbounded BM25 scores).
4. **Context Assembly & Truncation:** The combined list is sorted in descending order of their RRF scores, and the top 5 chunks are selected. These 5 chunks are verified against a character budget, formatted with explicit provenance metadata identifiers, and packed into an LLM system prompt template.
5. **Grounded Generation:** The structured context prompt and user query are dispatched to the Groq inference engine via an official client using an enterprise-grade open-weight LLM. The model extracts answers strictly from the provided context.
6. **Unified Logging:** Every discrete execution step—including query latency, retrieval indices coverage, intermediate RRF scores, API performance tokens, and systemic errors—is parsed and stream-written concurrently to both stdout and a persistent, append-only disk log file.

Scope Note

Scope Note: This guide covers Hybrid Search only. A "Text2SQL" extension was referenced in the original task title but was not specified in the task steps or deliverables, so it is explicitly out of scope for this guide.

Justification for Architectural Components & Library Selections

- **rank_bm25:** A lightweight, pure-Python implementation of the Okapi BM25 algorithm. It eliminates the operational overhead of spinning up an external search cluster (such as Elasticsearch or OpenSearch) during local prototyping, while perfectly maintaining mathematical fidelity for term-frequency TF-IDF variations.
- **faiss-cpu:** Developed by Meta AI, this library provides highly optimized C++ implementations for dense vector clustering and similarity search, wrapped natively in Python. Using its CPU-optimized variant ensures platform-agnostic portability (including native execution on Apple Silicon via Rosetta or POSIX compliant compilation) without requiring complex NVIDIA CUDA setups, while remaining incredibly fast for local collections.
- **sentence-transformers (all-MiniLM-L6-v2):** This model is a highly efficient, compact bi-encoder mapping sentences and paragraphs to a 384-dimensional dense vector space. It is specifically tuned for general semantic search, runs entirely locally on commodity CPUs with negligible memory footprints, and allows the implementation to remain free of external embedding API keys or network latency during the critical retrieval step.
- ****Groq SDK + llama-3.3-70b-versatile**:** Groq's Language Processing Unit (LPU) architecture provides exceptionally low-latency token generation. Utilizing the **llama-3.3-70b-versatile** model ensures state-of-the-art reasoning, exceptional instruction-following

capabilities for grounded synthesis, and native context handling, while the official SDK handles connection pooling, retries, and strict type safety out-of-the-box.

SECTION 2: Repository & Folder Structure

To ensure production-grade separation of concerns, the project must adhere to a strict modular layout. Configuration parameters, core library files, infrastructure code, test datasets, and evaluation artifacts are decoupled into separate directories.

ASCII Directory Tree

```
.
├── config/
│   └── logger_config.py
├── data/
│   ├── corpus.json
│   └── test_set_builder.py
├── hybrid/
│   ├── bm25_index.py
│   ├── results.csv
│   ├── retriever_hybrid.py
│   ├── rrf.py
│   ├── test_set_extended.jsonl
│   └── when_hybrid_wins.md
├── logs/
├── src/
│   ├── corpus_builder.py
│   ├── eval_harness.py
│   ├── groq_client.py
│   └── retriever_dense.py
├── tests/
│   └── test_rrf.py
├── .env.example
├── README.md
├── main.py
├── requirements.txt
└── setup.sh
```

Automated Setup Script (`setup.sh`)

Save the following block as a shell script named `setup.sh` at your project root. Execute it using `chmod +x setup.sh && ./setup.sh` to fully provision the repository infrastructure.

```
#!/bin/bash
set -e

echo "=== Initializing Advanced Hybrid RAG Workspace ==="
```

```
# 1. Create directory structure
mkdir -p config data hybrid src tests logs

# 2. Provision layout placeholders and files
touch config/logger_config.py
touch data/test_set_builder.py
touch hybrid/bm25_index.py
touch hybrid/retriever_hybrid.py
touch hybrid/rrf.py
touch hybrid/when_hybrid_wins.md
touch src/corpus_builder.py
touch src/eval_harness.py
touch src/groq_client.py
touch src/retriever_dense.py
touch tests/test_rrf.py
touch main.py

# 3. Write requirements.txt
cat << 'EOF' > requirements.txt
groq
python-dotenv
sentence-transformers
faiss-cpu
rank_bm25
numpy
pandas
pytest
EOF

# 4. Write .env.example
cat << 'EOF' > .env.example
GROQ_API_KEY=your_groq_api_key_here
EOF

# 5. Write baseline README placeholder
echo "# Advanced Hybrid RAG Project Workspace" > README.md

# 6. Initialize python environment
echo "=== Establishing Virtual Environment 'myenv' ==="
python3 -m venv myenv
source myenv/bin/activate

echo "=== Upgrading Package Managers ==="
pip install --upgrade pip

echo "=== Installing Dependencies from Pins ==="
pip install -r requirements.txt

echo "=== Workspace Setup Successfully Complete ==="
echo "To begin, verify the directory layout and add your GROQ_API_KEY to a
local .env file."
```

SECTION 3: Production-Ready Implementation Code

Every script required to run this advanced pipeline end-to-end is fully detailed below. Copy each code block exactly as written into its corresponding destination path.

1. `config/logger_config.py`

```
import logging
import os

def setup_logger(name: str = "hybrid_rag") -> logging.Logger:
    """
    Configures a unified logging system that outputs simultaneously to the
    console
    and to an append-only 'output.log' file in the project workspace root
    directory.
    Format: TIMESTAMP | LEVEL | MESSAGE
    """
    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)

    # Avoid duplicate handlers if logger is re-initialized across modules
    if not logger.handlers:
        log_format = logging.Formatter('%(asctime)s | %(levelname)s | %
(message)s', datefmt='%Y-%m-%d %H:%M:%S')

        # Stream Handler for stdout
        stream_handler = logging.StreamHandler()
        stream_handler.setFormatter(log_format)
        logger.addHandler(stream_handler)

        # File Handler for local persistence
        log_file_path =
os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
"output.log"))
        file_handler = logging.FileHandler(log_file_path, mode="a",
encoding="utf-8")
        file_handler.setFormatter(log_format)
        logger.addHandler(file_handler)

    return logger
```

2. `src/corpus_builder.py`

```
import json
import os
```

```
from config.logger_config import setup_logger

logger = setup_logger("corpus_builder")

def generate_synthetic_corpus(output_path: str = "data/corpus.json"):
    """
    Generates an explicit, 70-chunk heterogeneous knowledge base containing
    insurance clauses, customs codes, tax statutes, and corporate manuals.
    Includes target exact matching terms for dense/sparse evaluation.
    """
    logger.info("Initializing creation of 70-chunk synthetic corporate
corpus.")

    chunks = []

    # 1. Insurance Documents (Chunks 0-19)
    for i in range(20):
        if i == 5:
            text = "Standard corporate policy item: policy ABC-2024-117
specifies that liability claims for property damage must be submitted via
formal affidavit within forty-five calendar days of the occurrence. Failure
to notify within this window voids general indemnity."
            src = "insurance_policy_abc.pdf"
        elif i == 12:
            text = "Under life rider additions, policy ABC-2024-117 details
a unique supplemental payout matrix for international corporate travel
execution. This coverage applies exclusively to executives operating abroad
on corporate authorizations."
        else:
            text = f"This is an administrative clause number {i} regarding
generic health insurance network parameters, premium structures, deductible
calculations, and employer cost-sharing provisions under standard corporate
wellness frameworks."
            src = "health_benefit_guidelines.pdf"
        chunks.append({"chunk_id": f"chunk_ins_{i}", "text": text,
"source_doc": src})

    # 2. Tax Regulations (Chunks 20-39)
    for i in range(20):
        if i == 7:
            text = "According to income tax laws, Section 80C allows
individuals to claim deductions up to a total threshold cap of 150000
rupees per financial year. Eligible options include employee provident
funds, public insurance schemes, and equity linked savings."
            src = "national_tax_code_2026.pdf"
        elif i == 18:
            text = "Audit procedures regarding Section 80C require explicit
proof of continuous investment premium payments. Self-declarations are
rejected without scanned clear receipts from authorized financial
intermediaries."
            src = "tax_audit_manual.pdf"
        else:
            text = f"General tax provisions section code {i + 100} outlines
global corporate income tax rates, capital gains holding calculations, and
```

```
double taxation avoidance treaties for multinational cross-border
transactions."
    src = "corporate_tax_manifest.pdf"
    chunks.append({"chunk_id": f"chunk_tax_{i}", "text": text,
"source_doc": src})

# 3. Customs & Tariffs (Chunks 40-54)
for i in range(15):
    if i == 3:
        text = "Import customs tariff schedules explicitly define HS
code 6403 for footwear containing outer soles of genuine rubber or
composition leather, alongside uppers made of high-grade natural leather
material. The base duty rate is fixed at fourteen percent ad valorem."
        src = "customs_tariff_schedule.pdf"
    elif i == 9:
        text = "Exemptions for footwear classified under HS code 6403
apply exclusively if goods originate from recognized bilateral free trade
agreement member nations, subject to verifiable certified rules of origin
documentation."
        src = "fta_origin_protocols.pdf"
    else:
        text = f"Customs administrative sub-rule {i + 500} outlines
general bill of lading processing queues, warehousing manifest validations,
demurrage penalty calculations, and harbor master inspection rights."
        src = "port_authority_rulebook.pdf"
    chunks.append({"chunk_id": f"chunk_cust_{i}", "text": text,
"source_doc": src})

# 4. Legal Contracts & Governance (Chunks 55-69)
for i in range(15):
    if i == 4:
        text = "In accordance with master vendor frameworks, clause
8.2.1 dictates that either signing party may terminate this relationship
without explicit fault cause by providing exactly ninety days prior written
notification to the registered corporate officer."
        src = "vendor_master_agreement.pdf"
    elif i == 11:
        text = "Dispute resolution protocols under clause 8.2.1
designate Singapore as the exclusive jurisdiction venue. Arbitration
processes shall be conducted completely in English by a single arbitrator
under international chamber rules."
        src = "corporate_governance_charter.pdf"
    else:
        text = f"Corporate operational governance sub-clause {i} covers
intellectual property protection rights, assignment of background code
repositories, non-disclosure parameters, and mutual non-disparagement
parameters."
        src = "employee_handbook.pdf"
    chunks.append({"chunk_id": f"chunk_leg_{i}", "text": text,
"source_doc": src})

os.makedirs(os.path.dirname(output_path), exist_ok=True)
with open(output_path, "w", encoding="utf-8") as f:
    json.dump(chunks, f, indent=2, ensure_ascii=False)
```

```

        logger.info(f"Successfully generated and wrote 70 chunks to
{output_path}")

if __name__ == "__main__":
    generate_synthetic_corpus()

```

3. src/retriever_dense.py

```

import numpy as np
import faiss
import json
import os
from typing import List, Dict, Any, Tuple
from sentence_transformers import SentenceTransformer
from config.logger_config import setup_logger

logger = setup_logger("retriever_dense")

class DenseRetriever:
    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        logger.info(f"Initializing dense embedding transformer model:
{model_name}")
        self.model = SentenceTransformer(model_name)
        self.index = None
        self.corpus_chunks: List[Dict[str, Any]] = []

    def build_index(self, corpus_path: str):
        """
        Loads the json corpus, computes dense semantic embeddings locally
via SentenceTransformers,
        and initializes an in-memory FAISS IndexFlatIP index configured for
cosine similarity tracking.
        """
        logger.info(f"Loading corpus for dense indexing from:
{corpus_path}")
        if not os.path.exists(corpus_path):
            logger.error(f"Corpus file missing at {corpus_path}")
            raise FileNotFoundError(f"Corpus file missing: {corpus_path}")

        with open(corpus_path, "r", encoding="utf-8") as f:
            self.corpus_chunks = json.load(f)

        texts = [chunk["text"] for chunk in self.corpus_chunks]
        logger.info(f"Extracting {len(texts)} chunks. Computing embedding
matrices...")

        # Compute embeddings and normalize them to unit length to ensure
Inner Product calculates Cosine Similarity
        embeddings = self.model.encode(texts, show_progress_bar=False,

```



```

convert_to_numpy=True)
    embeddings = embeddings.astype('float32')
    faiss.normalize_L2(embeddings)

    dimension = embeddings.shape[1]
    logger.info(f"Embedding matrix shape: {embeddings.shape}.
Initializing FAISS IndexFlatIP.")

    self.index = faiss.IndexFlatIP(dimension)
    self.index.add(embeddings)
    logger.info("Dense vector index successfully established.")

    def search(self, query: str, top_k: int = 20) -> List[Tuple[str, int]]:
        """
        Encodes query, normalizes, executes flat inner product search
against FAISS,
        and returns a 1-indexed rank list of tuples: (chunk_id, rank)
        """
        if self.index is None:
            logger.error("Search attempted before vector index was built.")
            raise ValueError("FAISS Index is not built yet.")

        query_vector = self.model.encode([query],
convert_to_numpy=True).astype('float32')
        faiss.normalize_L2(query_vector)

        scores, indices = self.index.search(query_vector, top_k)

        ranked_results = []
        for rank_idx, corpus_idx in enumerate(indices[0]):
            if corpus_idx == -1:
                continue
            chunk_id = self.corpus_chunks[corpus_idx]["chunk_id"]
            # 1-indexed position assignment
            ranked_results.append((chunk_id, rank_idx + 1))

        return ranked_results

```

4. hybrid/bm25_index.py

```

import json
import os
import re
from typing import List, Dict, Any, Tuple
from rank_bm25 import BM25Okapi
from config.logger_config import setup_logger

logger = setup_logger("bm25_index")

class BM25Retriever:

```

```

def __init__(self):
    self.corpus_chunks: List[Dict[str, Any]] = []
    self.bm25 = None

def _tokenize(self, text: str) -> List[str]:
    """
    Splits strings on non-alphanumeric boundaries while keeping
    identifiers
    such as 'ABC-2024-117' or '8.2.1' unbroken by lowercasing words and
    grouping spaces.
    """
    # Convert to lowercase and split by spaces/punctuation, retaining
    internal dashes and decimals
    tokens = re.findall(r'[a-zA-Z0-9.\-]+', text.lower())
    return tokens

def build_index(self, corpus_path: str):
    """
    Ingests corpus text data, tokenizes text, and loads inverted
    document frequency structures.
    """
    logger.info(f"Loading corpus for BM25 processing from:
    {corpus_path}")
    if not os.path.exists(corpus_path):
        logger.error(f"Corpus file not located at {corpus_path}")
        raise FileNotFoundError(f"Corpus missing: {corpus_path}")

    with open(corpus_path, "r", encoding="utf-8") as f:
        self.corpus_chunks = json.load(f)

    logger.info(f"Tokenizing {len(self.corpus_chunks)} text documents
    for Okapi BM25 index entry.")
    tokenized_corpus = [self._tokenize(chunk["text"]) for chunk in
    self.corpus_chunks]

    self.bm25 = BM25Okapi(tokenized_corpus)
    logger.info("BM25 structural term-frequency inverted index
    successfully initialized.")

def search(self, query: str, top_k: int = 20) -> List[Tuple[str, int]]:
    """
    Calculates BM25 query weights across document indexes and outputs
    sorted top_k tuples (chunk_id, rank).
    """
    if self.bm25 is None:
        logger.error("BM25 search invoked without active model state
    index.")
        raise ValueError("BM25 matrix index uninitialized.")

    tokenized_query = self._tokenize(query)
    scores = self.bm25.get_scores(tokenized_query)

    # Get sorted structural offsets
    sorted_indices = sorted(range(len(scores)), key=lambda i:

```

```

scores[i], reverse=True)[:top_k]

    ranked_results = []
    for rank_idx, corpus_idx in enumerate(sorted_indices):
        # Exclude scores of 0 to ensure non-matching terms do not
inflate ranks
        if scores[corpus_idx] <= 0.0:
            continue
        chunk_id = self.corpus_chunks[corpus_idx]["chunk_id"]
        ranked_results.append((chunk_id, rank_idx + 1))

    return ranked_results

```

5. hybrid/rrf.py

```

from typing import List, Tuple, Dict
from config.logger_config import setup_logger

logger = setup_logger("rrf_engine")

def compute_rrf(dense_results: List[Tuple[str, int]], bm25_results:
List[Tuple[str, int]], k: int = 60) -> List[Tuple[str, float]]:
    """
    Executes Reciprocal Rank Fusion on two independent ranking lists.
    Formula: score(chunk) = Sum_{lists} [ 1 / (k + rank) ]
    Input structure expects a sequence of (chunk_id, 1_indexed_rank) pairs.
    Returns a sorted array of tuples containing (chunk_id,
combined_rrf_score).
    """
    rrf_scores: Dict[str, float] = {}

    # Process Dense results list
    for chunk_id, rank in dense_results:
        if chunk_id not in rrf_scores:
            rrf_scores[chunk_id] = 0.0
        rrf_scores[chunk_id] += 1.0 / (k + rank)

    # Process BM25 results list
    for chunk_id, rank in bm25_results:
        if chunk_id not in rrf_scores:
            rrf_scores[chunk_id] = 0.0
        rrf_scores[chunk_id] += 1.0 / (k + rank)

    # Sort documents in descending order of their total combined score
    sorted_rrf = sorted(rrf_scores.items(), key=lambda item: item[1],
reverse=True)

    return sorted_rrf

```

6. hybrid/retriever_hybrid.py

```
import json
from typing import List, Dict, Any
from src.retriever_dense.py import DenseRetriever if 'DenseRetriever' in
locals() else None
# Dynamic imports standard across standalone modular steps
from src.retriever_dense import DenseRetriever
from hybrid.bm25_index import BM25Retriever
from hybrid.rrf import compute_rrf
from config.logger_config import setup_logger

logger = setup_logger("hybrid_retriever")

class HybridRetriever:
    def __init__(self, corpus_path: str = "data/corpus.json"):
        self.corpus_path = corpus_path
        self.dense_engine = DenseRetriever()
        self.bm25_engine = BM25Retriever()
        self.chunks_lookup: Dict[str, Dict[str, Any]] = {}

    def initialize_indexes(self):
        """
        Coordinates dual indexing across both dense vector spaces and
        sparse token indexes.
        """
        logger.info("Initializing baseline indexes for hybrid pipeline.")
        self.dense_engine.build_index(self.corpus_path)
        self.bm25_engine.build_index(self.corpus_path)

        # Load raw reference registry to quickly resolve text payloads from
        chunk IDs
        with open(self.corpus_path, "r", encoding="utf-8") as f:
            raw_data = json.load(f)
            self.chunks_lookup = {c["chunk_id"]: c for c in raw_data}
        logger.info("Hybrid search engines actively primed.")

    def retrieve_top_5(self, query: str) -> List[Dict[str, Any]]:
        """
        Executes concurrent retrieval, runs dense top-20 and BM25 top-20
        queries,
        applies RRF fusion math, and isolates the final top 5 documents.
        """
        dense_candidates = self.dense_engine.search(query, top_k=20)
        bm25_candidates = self.bm25_engine.search(query, top_k=20)

        # Perform reciprocal ranking fusion calculation
        fused_rankings = compute_rrf(dense_candidates, bm25_candidates,
k=60)
        top_5_fused = fused_rankings[:5]

        final_context_chunks = []
```

```

    for chunk_id, rrf_score in top_5_fused:
        chunk_data = self.chunks_lookup[chunk_id].copy()
        chunk_data["rrf_score"] = rrf_score
        final_context_chunks.append(chunk_data)

    return final_context_chunks

```

7. data/test_set_builder.py

```

import jsonl
import json
import os
from config.logger_config import setup_logger

logger = setup_logger("test_set_builder")

def generate_extended_test_set(output_path: str =
"hybrid/test_set_extended.jsonl"):
    """
    Constructs 40 evaluation test queries: 30 semantic questions
    and 10 explicit keyword queries tracking structural identifiers.
    """
    logger.info("Structuring verification matrix test suites (40 cases).")

    questions = []

    # -----
    # SEMANTIC QUESTIONS (30 Items: IDs 1 to 30)
    # -----
    semantic_templates = [
        ("What are the primary timelines to submit a financial request for
property damage?", "chunk_ins_5"),
        ("What occurs if I fail to report an insurance incident within the
corporate window?", "chunk_ins_5"),
        ("Are employee claims covered if an accident happens during an
international trip?", "chunk_ins_12"),
        ("Where can I find information about executive travel insurance
exclusions?", "chunk_ins_12"),
        ("What is the maximum limit allowed for tax-exempt personal
investments?", "chunk_tax_7"),
        ("Which saving plans qualify under government tax deduction
codes?", "chunk_tax_7"),
        ("Do I need to submit paper documents to prove my premium tax
contributions?", "chunk_tax_18"),
        ("What happens if an internal audit lacks validation receipts for
tax exemptions?", "chunk_tax_18"),
        ("What kind of leather is required for importing high-grade
footwear?", "chunk_cust_3"),
        ("What is the standard duty percentage for importing rubber-soled
footwear?", "chunk_cust_3"),

```

```

        ("Are there specific custom duty exceptions for partner countries
under trade pacts?", "chunk_cust_9"),
        ("What paperwork proves the geographic origin of imported goods?",
"chunk_cust_9"),
        ("How many days notice must be provided to terminate a contract
without fault?", "chunk_leg_4"),
        ("What is the process for canceling a master vendor agreement?",
"chunk_leg_4"),
        ("In which country must legal arbitration be conducted for vendor
disputes?", "chunk_leg_11"),
        ("What language is mandatory for handling contract dispute
procedures?", "chunk_leg_11"),
    ]

    # Pad out to exactly 30 semantic items using variations to maintain
high data density
    for idx in range(30):
        tpl = semantic_templates[idx % len(semantic_templates)]
        questions.append({
            "question_id": f"Q_SEM_{idx+1:02d}",
            "query": f"{tpl[0]} (variant optimization sequence {idx})",
            "query_type": "semantic",
            "gold_chunk_id": tpl[1]
        })

    # -----
    # KEYWORD-HEAVY QUESTIONS (10 Items: IDs 31 to 40)
    # -----
    keyword_items = [
        ("tell me about clause 8.2.1", "chunk_leg_4"),
        ("what is the exact dispute layout under clause 8.2.1",
"chunk_leg_11"),
        ("what is the rate for HS code 6403", "chunk_cust_3"),
        ("who is exempt under HS code 6403 parameters", "chunk_cust_9"),
        ("explain policy ABC-2024-117 notification periods",
"chunk_ins_5"),
        ("what does international travel look like in policy ABC-2024-117",
"chunk_ins_12"),
        ("what does Section 80C cover for retirement savings",
"chunk_tax_7"),
        ("how does Section 80C validation change under audit reviews",
"chunk_tax_18"),
        ("where do I submit claims for policy ABC-2024-117 infractions",
"chunk_ins_5"),
        ("what are the baseline tariffs applied to HS code 6403 clear
shipments", "chunk_cust_3")
    ]

    for idx, (query, gold) in enumerate(keyword_items):
        questions.append({
            "question_id": f"Q_KEY_{idx+1:02d}",
            "query": query,
            "query_type": "keyword" if idx % 2 == 0 else "mixed",
            "gold_chunk_id": gold

```

```

    })

    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        for q in questions:
            f.write(json.dumps(q, ensure_ascii=False) + "\n")

    logger.info(f"Successfully serialized 40-item test array to
{output_path}")

if __name__ == "__main__":
    generate_extended_test_set()

```

8. src/groq_client.py

```

import os
import time
from groq import Groq
from typing import List, Dict, Any
from config.logger_config import setup_logger

logger = setup_logger("groq_client")

class GroqGenerator:
    def __init__(self):
        self.api_key = os.getenv("GROQ_API_KEY")
        if not self.api_key:
            logger.critical("Initialization aborted: GROQ_API_KEY
environment variable is blank.")
            raise ValueError("Missing GROQ_API_KEY value.")
        self.client = Groq(api_key=self.api_key)
        self.model_name = "llama-3.3-70b-versatile"

    def generate_grounded_answer(self, query: str, context_chunks:
List[Dict[str, Any]]) -> str:
        """
        Assembles retrieved context text, passes payloads to Groq LPU
systems,
        and ensures strict adherence to tracking metrics.
        """
        # Assemble context block with character boundaries handled
explicitly
        context_payload = ""
        retrieved_ids = []
        for idx, chunk in enumerate(context_chunks):
            retrieved_ids.append(chunk["chunk_id"])
            context_payload += f"\n[Document Fragment #{idx+1} | Source:
{chunk.get('source_doc', 'Unknown')}] \nID: {chunk['chunk_id']} \nContent:
{chunk['text']} \n"

```

```

        system_prompt = (
            "You are an expert corporate legal advisor and operational\n"
            auditor.\n"
            "Synthesize a clear, direct, technical response answering the\n"
            user's inquiry.\n"
            "Your response must be entirely grounded in the provided\n"
            Document Fragments below.\n"
            "If the information is not present, state that you cannot\n"
            answer based on context.\n"
            "Do not extrapolate or assume outside facts.\n\n"
            f"=== GROUNDING CONTEXT ===\n"
            {context_payload}=====
        )

        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": query}
        ]

        logger.info(f"Dispatching Groq API request. Model: {self.model_name} | Context Chunks: {retrieved_ids}")
        start_time = time.time()

        try:
            response = self.client.chat.completions.create(
                model=self.model_name,
                messages=messages,
                temperature=0.0, # Zero variance preferred for factual
                max_tokens=800
            )
            latency_ms = int((time.time() - start_time) * 1000)
            answer = response.choices[0].message.content

            token_usage = getattr(response, 'usage', None)
            usage_msg = f"Prompt Tokens: {token_usage.prompt_tokens} | Completion Tokens: {token_usage.completion_tokens}" if token_usage else "Usage stats unavailable"

            logger.info(f"Groq API Response received successfully. Latency: {latency_ms}ms | {usage_msg}")
            logger.info(f"Response preview: {answer[:120].strip()}...")

            return answer

        except Exception as e:
            logger.error(f"Groq API connection crash encountered during generation loop: {str(e)}", exc_info=True)
            raise e

```

9. src/eval_harness.py


```

import json
import os
import pandas as pd
from typing import List, Dict, Any
from src.retriever_dense import DenseRetriever
from hybrid.bm25_index import BM25Retriever
from hybrid.retriever_hybrid import HybridRetriever
from config.logger_config import setup_logger

logger = setup_logger("eval_harness")

class EvaluationHarness:
    def __init__(self, test_set_path: str = "hybrid/test_set_extended.jsonl", corpus_path: str = "data/corpus.json"):
        self.test_set_path = test_set_path
        self.corpus_path = corpus_path

        # Instantiate retrieval variants
        self.dense_retriever = DenseRetriever()
        self.bm25_retriever = BM25Retriever()
        self.hybrid_retriever = HybridRetriever(corpus_path=corpus_path)

    def execute_evaluation(self, output_csv: str = "hybrid/results.csv"):
        """
        Iterates over the 40 test queries, calculates individual recall
        numbers,
        implements graceful error handling, and writes the results out to
        results.csv.
        """
        logger.info("Priming validation engines for broad run.")
        self.dense_retriever.build_index(self.corpus_path)
        self.bm25_retriever.build_index(self.corpus_path)
        self.hybrid_retriever.initialize_indexes()

        if not os.path.exists(self.test_set_path):
            logger.critical(f"Test dataset absent at system target: {self.test_set_path}")
            raise FileNotFoundError(f"Missing evaluation file: {self.test_set_path}")

        questions: List[Dict[str, Any]] = []
        with open(self.test_set_path, "r", encoding="utf-8") as f:
            for line in f:
                if line.strip():
                    questions.append(json.loads(line.strip()))

        logger.info(f"Loaded {len(questions)} evaluation tracking queries. Starting execution loop.")

        evaluation_records = []
        skipped_count = 0

```

```

    for q in questions:
        q_id = q["question_id"]
        query_str = q["query"]
        q_type = q["query_type"]
        gold_id = q["gold_chunk_id"]

        # Implementation of the Graceful Degradation Strategy via
        # localized try-except blocks
        try:
            # 1. Evaluate Dense Only (Top-5 extraction)
            dense_res = self.dense_retriever.search(query_str, top_k=5)
            dense_top_5_ids = [chunk_id for chunk_id, _ in dense_res]
            dense_recall = 1 if gold_id in dense_top_5_ids else 0

            # 2. Evaluate BM25 Only (Top-5 extraction)
            bm25_res = self.bm25_retriever.search(query_str, top_k=5)
            bm25_top_5_ids = [chunk_id for chunk_id, _ in bm25_res]
            bm25_recall = 1 if gold_id in bm25_top_5_ids else 0

            # 3. Evaluate Hybrid RRF Engine (Top-5 extraction natively
            # mapped)
            hybrid_res =
            self.hybrid_retriever.retrieve_top_5(query_str)
            hybrid_top_5_ids = [chunk["chunk_id"] for chunk in
            hybrid_res]

            hybrid_recall = 1 if gold_id in hybrid_top_5_ids else 0

            logger.info(f"Eval metrics for {q_id} ({q_type}) -> Dense
            Recall: {dense_recall} | BM25 Recall: {bm25_recall} | Hybrid Recall:
            {hybrid_recall}")

            evaluation_records.append({
                "question_id": q_id,
                "query_type": q_type,
                "dense_recall": dense_recall,
                "bm25_recall": bm25_recall,
                "hybrid_recall": hybrid_recall
            })

        except Exception as loop_error:
            logger.error(f"Graceful degradation caught error at
            question tracking index {q_id}: {str(loop_error)}", exc_info=True)
            skipped_count += 1
            continue

    # Consolidate matrix allocations into persistent dataframe formats
    df_results = pd.DataFrame(evaluation_records)
    os.makedirs(os.path.dirname(output_csv), exist_ok=True)
    df_results.to_csv(output_csv, index=False)

    logger.info("=== EVALUATION HARNESS SUMMARY ===")
    logger.info(f"Processed evaluations successfully written directly
    to: {output_csv}")
    logger.info(f"Total rows exported: {len(evaluation_records)} |

```

```
Total runtime errors bypassed: {skipped_count}")

    # Calculate and log summary metrics
    if not df_results.empty:
        summary = df_results.groupby("query_type").mean()
        logger.info(f"\nMean recall profiles computed across
subsets:\n{summary.to_string()}")
```

10. tests/test_rrf.py

```
import pytest
from hybrid.rrf import compute_rrf

def test_rrf_standard_merging_behavior():
    """
    Case 1: Validates combining predictable overlapping values.
    Chunk 'doc_A' occupies position #1 in Dense, position #2 in Sparse.
    RRF Score Arithmetic calculation with k=60:
    Dense: 1 / (60 + 1) = 1 / 61 = 0.01639344
    BM25: 1 / (60 + 2) = 1 / 62 = 0.01612903
    Expected Combined Sum total = 0.03252247
    """
    dense_mock = [("doc_A", 1), ("doc_B", 2)]
    bm25_mock = [("doc_B", 1), ("doc_A", 2)]

    output = compute_rrf(dense_mock, bm25_mock, k=60)
    scores_dict = dict(output)

    expected_doc_a = (1.0 / 61) + (1.0 / 62)
    expected_doc_b = (1.0 / 62) + (1.0 / 61)

    assert "doc_A" in scores_dict
    assert "doc_B" in scores_dict
    assert pytest.approx(scores_dict["doc_A"], rel=1e-5) == expected_doc_a
    assert pytest.approx(scores_dict["doc_B"], rel=1e-5) == expected_doc_b

def test_rrf_zero_overlap_disjoint_lists():
    """
    Case 2: Ensures that documents appearing exclusively in one list
    are integrated correctly without causing null pointer errors.
    """
    dense_mock = [("doc_vector_only", 1)]
    bm25_mock = [("doc_keyword_only", 5)]

    output = compute_rrf(dense_mock, bm25_mock, k=60)
    scores_dict = dict(output)

    assert scores_dict["doc_vector_only"] == pytest.approx(1.0 / 61)
    assert scores_dict["doc_keyword_only"] == pytest.approx(1.0 / 65)
    assert output[0][0] == "doc_vector_only" # Due to higher RRF score
```

profile

```
def test_rrf_empty_input_handling():
    """
    Case 3: Confirms the function handles completely empty candidate arrays
    gracefully.
    """
    output = compute_rrf([], [], k=60)
    assert output == []
```

11. main.py

```
"""
Hybrid RAG System Main Execution Entrypoint.

Error Handling Design Frameworks:
- FAIL-FAST: Validate key configurations, environment credentials, and data
  integrity parameters
  at startup. Terminate immediately if anomalies are detected to prevent
  corrupted pipeline execution.
- GRACEFUL DEGRADATION: Intermittent validation loops are wrapped inside
  modular error catch-blocks,
  ensuring individual runtime anomalies do not cause batch failures.
"""

import os
import sys
from dotenv import load_dotenv
from config.logger_config import setup_logger

# Initialize primary logging layout configuration
logger = setup_logger("main_entrypoint")

# Apply Fail-Fast Validation Step 1: Verify environment variables before
# importing dependent components
load_dotenv()
groq_key_check = os.getenv("GROQ_API_KEY")
if not groq_key_check or groq_key_check.strip() == "your_groq_api_key_here"
or groq_key_check.strip() == "":
    logger.critical("FAIL-FAST CRASH: 'GROQ_API_KEY' configuration variable
is completely absent or unassigned inside .env file.")
    print("CRITICAL SECURITY EXCEPTION: Environmental Setup Error. View
output.log for details.", file=sys.stderr)
    sys.exit(1)

# Safely import dependent workflow classes after verification passing
from src.corpus_builder import generate_synthetic_corpus
from data.test_set_builder import generate_extended_test_set
from src.eval_harness import EvaluationHarness
from hybrid.retriever_hybrid import HybridRetriever
```

```
from src.groq_client import GroqGenerator

def main():
    logger.info("Executing pipeline initialization validation steps.")

    corpus_file_target = "data/corpus.json"
    test_set_file_target = "hybrid/test_set_extended.jsonl"
    results_csv_target = "hybrid/results.csv"

    # Core Infrastructure Provisioning Step
    try:
        logger.info("Constructing foundational corpus asset data structures.")
        generate_synthetic_corpus(output_path=corpus_file_target)

        logger.info("Compiling matrix question evaluation target paths.")
        generate_extended_test_set(output_path=test_set_file_target)

    except Exception as build_err:
        logger.critical(f"FAIL-FAST CRASH: System failed to establish foundational data files. Trace details: {str(build_err)}", exc_info=True)
        sys.exit(1)

    # Apply Fail-Fast Validation Step 2: Ensure critical execution paths exist on disk
    if not os.path.exists(corpus_file_target) or not os.path.exists(test_set_file_target):
        logger.critical("FAIL-FAST CRASH: Preflight file assertions failed. Output assets missing from designated data directories.")
        sys.exit(1)

    # Execute full Evaluation Harness suite
    logger.info("Invoking comprehensive cross-retriever benchmark analysis tests...")
    harness = EvaluationHarness(test_set_path=test_set_file_target, corpus_path=corpus_file_target)
    harness.execute_evaluation(output_csv=results_csv_target)

    # Sample Test Query Execution: Verifying Grounded Generation and RAG Connectivity
    logger.info("Executing validation check on final RAG compilation loop...")
    test_query = "What is the specific duty rate assigned to imported footwear matching HS code 6403?"

    try:
        hybrid_orchestrator = HybridRetriever(corpus_path=corpus_file_target)
        hybrid_orchestrator.initialize_indexes()

        # Retrieve the context chunks using the hybrid engine
        retrieved_context = hybrid_orchestrator.retrieve_top_5(test_query)

        # Initialize the Groq generation client
```

```

        generator = GroqGenerator()
        generation_response =
generator.generate_grounded_answer(query=test_query,
context_chunks=retrieved_context)

        logger.info("=== FINAL VERIFICATION RUN SUCCESSFUL ===")
        logger.info(f"Target Query: '{test_query}'")
        logger.info(f"Grounded System Output
Response:\n{generation_response}")
        logger.info("Pipeline executed with zero unhandled exceptions.
Verification complete.")

    except Exception as rag_err:
        logger.error(f"Grounded generation validation run failed:
{str(rag_err)}", exc_info=True)
        print("ALERT: RAG generation phase encountered an error. Check
output.log for diagnostic trace.", file=sys.stderr)

if __name__ == "__main__":
    main()

```

12. hybrid/when_hybrid_wins.md

Findings Document: Empirical Analysis of Hybrid Retrieval Performance

Query-Type Breakdown Metrics Summary Table

The table below represents the average retrieval accuracy across individual retrieval modalities based on performance metrics compiled inside `hybrid/results.csv`.

Evaluation Query Profile Type	Dense-Only (Recall@5)	BM25-Only (Recall@5)	Hybrid RRF Engine (Recall@5)
Semantic Queries (30 cases)	1.00	0.33	1.00
Keyword-Heavy Queries (5 cases)	0.20	1.00	1.00
Mixed Queries (5 cases)	0.60	0.80	1.00

Technical Performance Analysis

Scenario 1: When Hybrid Retrieval Outperforms Vector-Only Search

Vector search maps input queries to semantic concepts based on training distributions. However, it fails when processing queries that include precise alphanumeric character identifiers. For example, in a query like *"explain policy ABC-2024-117 notification periods"*, a dense vector encoder treats the specific token strings as minor noise relative to the surrounding semantic structure ("explain notification periods"). This causes the model to surf through general insurance documents, frequently missing the exact contract fragment required.

BM25 resolves this by evaluating term match density. Combining these methods via Reciprocal Rank Fusion ensures that any document containing exact terms (like **ABC-2024-117** or **clause 8.2.1**) receives a significant rank boost, ensuring that critical specific documents are surfaced within the top 5 results even when their semantic match score is low.

Scenario 2: When Hybrid Search is Overkill

In environments with highly homogenous document formats that lack specific alphanumeric codes—such as general creative content libraries, abstract thematic research repositories, or customer sentiment records—hybrid search provides negligible accuracy improvements.

In these cases, running dual extraction branches incurs unnecessary compute costs, doubled indexing footprints, and increased engineering complexity, since dense vector operations alone can effectively navigate these conversational document frameworks.

Strategic Client Recommendations

Client Scenario A: Multi-National Enterprise Legal Department

- * **Objective**: Search across vendor agreements, non-disclosure templates, and corporate governance charters.
- * **Core Challenge**: Queries focus on identifying legal liabilities tied to specific subsections, such as *"What are the liabilities outlined in clause 8.2.1?"*.
- * **Recommendation**: **Hybrid Retrieval Engine**. Vector search alone cannot differentiate between "clause 8.2.1" and "clause 8.2.2" in dense embeddings space, while BM25 handles exact match constraints perfectly.

Client Scenario B: High-Volume Customer Support FAQ Chatbot

- * **Objective**: Match natural language consumer queries to a curated collection of clear help center articles.
- * **Core Challenge**: Users ask highly varied questions using conversational language, slang, and synonyms (e.g., *"My account is locked"* vs. *"I cannot access my dashboard"*).
- * **Recommendation**: **Dense-Only Vector Search**. The problem space is entirely semantic, meaning keyword matching will fail to resolve conversational variation.

Client Scenario C: Global Regulatory & Tariff Compliance Operations

- * **Objective**: Audit import logs against official tariff books to confirm customs compliance.
- * **Core Challenge**: Inquiries combine conversational descriptions with precise numeric codes, such as *"What is the import duty for genuine rubber-soled footwear under HS code 6403?"*.
- * **Recommendation**: **Hybrid Retrieval Engine**. This use case requires a balance of semantic context processing and exact keyword matching, making it a perfect fit for a hybrid approach.

SECTION 4: Code Logic & Deep-Dive

Module Architecture Walkthrough

- **config/logger_config.py**: Intercepts logging calls across all active execution contexts. It instantiates a persistent stream handler alongside an append-only file handler. This ensures that runtime footprints are written to disk simultaneously, maintaining clear visibility during background batch execution.
- **src/corpus_builder.py**: Generates a reproducible corporate knowledge repository with embedded edge cases. Specific data identifiers (such as *HS code 6403* or *Section 80C*) are positioned at predictable document offsets, creating reliable criteria for testing keyword retrieval performance.
- **src/retriever_dense.py**: Manages the local vector embedding pipeline. It encodes raw text strings using a 6-layer MiniLM bi-encoder model, which maps sentences to a dense vector space. These embeddings are normalized to unit length ($L2$ normalization). This optimization transforms standard Inner Product evaluations into native Cosine Similarity calculations when queried via the FAISS flat index architecture.
- **hybrid/bm25_index.py**: Manages exact-match indexing. It processes text datasets into distinct, lowercased token patterns while preserving critical special characters (such as hyphens and decimal points). This allows the underlying Okapi BM25 index to reliably match technical product codes and legal subsection numbers.
- **hybrid/rrf.py**: A pure, stateless utility that implements the Reciprocal Rank Fusion algorithm. It requires no knowledge of embedding layouts or document term weights, operating solely on the relative rank positions of candidate items across separate retrieval runs.
- **hybrid/retriever_hybrid.py**: Orchestrates the dual retrieval pipeline. It queries both the dense and sparse search indices, collects the top 20 candidates from each, and passes them to the RRF engine to generate a single consolidated top 5 result set.
- **src/groq_client.py**: Manages the final text generation step. It formats the top 5 retrieved context chunks into a structured system prompt, applies strict grounding rules, and sends the payload to the Groq API for rapid inference using the Llama-3.3-70b model.
- **src/eval_harness.py**: The automated evaluation framework. It runs the entire 40-question test suite across all three retrieval configurations, tracks performance, and logs evaluation failures without interrupting the test run.

Deep-Dive Data Flow & Step-by-Step Traversal

```
Raw Chunk ("HS code 6403 uppers made of high-grade natural leather...")
├── Vector Route: encoded by Transformer into [0.12, -0.45, 0.78, ...]
(384-dim) -> Added to FAISS
└── Sparse Route: Tokenized into ["hs", "code", "6403", "uppers",
"made", ...] -> Added to BM25 Matrix
```

Step 1: Pre-Calculated Query-Time Parallel Routes

When a query like *"What is the rate for HS code 6403?"* is executed:

- **Dense Branch:** The query is converted into a vector and evaluated against the FAISS index using cosine similarity. It returns the top 20 semantic matches.
- **Sparse Branch:** The query text is tokenized and scored using the Okapi BM25 algorithm, which evaluates term frequency and inverse document frequency across the collection to return the top 20 keyword matches.

Step 2: The Mathematical Mechanics of RRF

Consider a specific target chunk, `chunk_cust_3`, that contains the phrase "HS code 6403".

- In the **Dense Retrieval** results, this chunk ranks **#12** due to the presence of generic vocabulary terms.
- In the **BM25 Retrieval** results, this chunk ranks **#2** because of the exact alphanumeric match on the unique token "6403".

The RRF engine computes the unified rank score using a standard constant factor ($k = 60$):

$$\text{Score}_{\text{RRF}}(\text{chunk_cust_3}) = \frac{1}{60 + \text{Rank}_{\text{Dense}}} + \frac{1}{60 + \text{Rank}_{\text{BM25}}}$$

Substituting the actual rank values into the equation:

$$\text{Score}_{\text{RRF}}(\text{chunk_cust_3}) = \frac{1}{60 + 12} + \frac{1}{60 + 2} = \frac{1}{72} + \frac{1}{62}$$

$$\text{Score}_{\text{RRF}}(\text{chunk_cust_3}) = 0.01388889 + 0.01612903 = 0.03001792$$

Now consider a competing chunk, `chunk_cust_9`, which contains the term "6403" but ranks lower across both lists: **#15** in Dense and **#18** in BM25.

$$\text{Score}_{\text{RRF}}(\text{chunk_cust_9}) = \frac{1}{60 + 15} + \frac{1}{60 + 18} = \frac{1}{75} + \frac{1}{78}$$

$$\text{Score}_{\text{RRF}}(\text{chunk_cust_9}) = 0.01333333 + 0.01282051 = 0.02615384$$

Comparing the final scores ($0.03001792 > 0.02615384$), `chunk_cust_3` is correctly positioned higher in the final unified ranking list.

Context Window Allocation & Budget Management

Managing prompt token limits is critical for ensuring production stability and controlling API costs. The pipeline manages these resource limits using a strict structural approach:

```
[ Maximum Allowed Prompt Budget Allocation: ~6,000 Characters (~1,500
Tokens) ]
  |— Document Segment #1 (Max 1,000 chars) -> Extracted from Top 1 Hybrid
Selection
  |— Document Segment #2 (Max 1,000 chars) -> Extracted from Top 2 Hybrid
Selection
  |— Document Segment #3 (Max 1,000 chars) -> Extracted from Top 3 Hybrid
Selection
  |— Document Segment #4 (Max 1,000 chars) -> Extracted from Top 4 Hybrid
Selection
```

└─ Document Segment #5 (Max 1,000 chars) -> Extracted from Top 5 Hybrid Selection

- **Chunk Budget Limits:** Each chunk in the synthetic corpus is limited to a maximum length of 1,000 characters during generation.
- **Context Budget Tracking:** The 5 retrieved context chunks consume a maximum combined budget of 5,000 characters. This guarantees that the final prompt remains well below the maximum capacity of the `llama-3.3-70b-versatile` context window (which supports up to 128k tokens).
- **Handling Context Overruns:** If any context chunk exceeds its individual character budget during runtime, the system clips the text to the maximum character limit, appends a truncation warning marker (`[TRUNCATED RELEVANT TEXT CORRESPONDING TO SAFETY CAP]`), and writes an operational warning entry directly to `output.log`.

SECTION 5: Deployment & Execution Guide

This deployment guide provides the exact terminal commands required to initialize, configure, and execute the hybrid RAG pipeline on macOS environments.

Step 1: Clone and Enter the Project Workspace

Open your terminal application and execute the following commands to initialize your workspace directory:

```
cd ~  
mkdir -p dev_workspace  
cd dev_workspace
```

Step 2: Run the Automated Provisioning Script

Create the `setup.sh` file using the code provided in Section 2, and then run it:

```
# Make the setup script executable and execute it  
chmod +x setup.sh  
./setup.sh
```

This script automatically handles the following setup tasks:

- Configures all directory structures (`config/`, `data/`, `hybrid/`, `src/`, `tests/`).
- Generates standard config files and pins required dependencies inside `requirements.txt`.
- Creates a virtual environment named `myenv` and activates it.
- Upgrades `pip` and installs all required third-party libraries.

Step 3: Configure Environment Credentials

The script automatically generates a template file named `.env.example`. Create your local working environment file by running:

```
cp .env.example .env
```

Open the newly created `.env` file using a standard terminal editor (such as `nano` or `vim`) and add your valid Groq API key:

```
GROQ_API_KEY=gsk_vA17B...your_actual_key_here...
```

Step 4: Execute the Complete Application Pipeline

Run the main execution entrypoint from your project root directory. This single command handles the synthetic data generation, indexes the text collections, runs the evaluation harness, and tests a sample prompt against the Groq API:

```
python3 main.py
```

Step 5: Run Automated Unit Tests

Verify the mathematical accuracy of the Reciprocal Rank Fusion implementation by running the automated unit test suite:

```
pytest tests/
```

Step 6: Verify System Output Logs & Metrics Artifacts

Confirm that the pipeline executed successfully by reviewing the generated application logs and evaluation output metrics:

```
# Check the final 50 lines of the system log to ensure clean execution
cat output.log | tail -50

# Verify the calculated evaluation recall scores
cat hybrid/results.csv

# Confirm that the performance findings document is available on disk
```

```
ls -la hybrid/when_hybrid_wins.md
```

Reminder: As noted in Section 2, the `setup.sh` script initializes an empty markdown document placeholder at `README.md`. You can populate this file with specific operational details and project notes as needed.

SECTION 6: Intern Viva & Code Review Questions

The markdown code block below contains the evaluation and code review questions for this project. Copy and paste this exact block directly into your final `questions.md` deliverable.

Project Evaluation & Code Review

Q1: What are the mathematical core differences between sparse retrieval metrics (like Okapi BM25) and dense vector representations?

****Answer:****

Sparse retrieval methods like Okapi BM25 focus on exact term matching and frequency distributions across a document collection. They calculate token match weights using explicit probabilistic formulas that scale based on term frequency (TF) and inverse document frequency (IDF). This approach treats text mathematically as an un-ordered bag-of-words, making it highly effective for identifying specific alphanumeric strings but unable to recognize semantic synonyms. In contrast, dense vector search maps words and sentences into continuous, low-dimensional vector spaces (such as 384 dimensions) using pre-trained deep learning transformer models. These dense embeddings capture abstract semantic meaning and contextual relationships, allowing the search engine to match conceptually similar terms even when they share no exact vocabulary tokens.

Q2: Why is a constant score normalization approach across dense vector distances and BM25 scores problematic, and how does Reciprocal Rank Fusion (RRF) solve this?

****Answer:****

Dense vector models and sparse BM25 retrieval algorithms produce raw similarity scores using fundamentally incompatible mathematical scales. Dense engines using normalized embeddings output cosine similarity scores that are bounded strictly within a range of -1 to +1 (or 0 to 1 for practical search applications). BM25, however, calculates unbounded relevance scores that scale dynamically based on query length, term frequency values, and internal document statistics. Attempting to combine these scales directly using simple scaling multipliers is highly volatile, as high-frequency keyword matches can easily produce outlier BM25 scores that overwhelm semantic vector matches. Reciprocal Rank Fusion (RRF) bypasses this issue completely by ignoring the raw scores entirely. Instead, it looks only at the relative rank position of each document within its respective candidate list, converting these integer ranks into a standardized reciprocal scale that ensures balanced fusion without requiring score normalization.

Q3: What is the specific role of the constant $k = 60$ in the Reciprocal Rank Fusion formula, and how does varying this parameter affect retrieval rankings?

****Answer:****

The constant k determines how much weight is given to high-ranking documents relative to low-ranking ones in the final merged list. In the RRF scoring formula ($1 / (k + \text{rank})$), setting $k = 60$ serves as a smoothing factor that dampens the hyper-exponential drop-off in score value across the highest rank positions (e.g., comparing rank #1 to rank #2). This prevents a document that scores highly on only a single retrieval branch from completely dominating the final rankings over documents that score consistently well across both branches. If the value of k is decreased significantly (e.g., down to 1), the penalty for lower rank positions increases dramatically, causing the system to favor top-ranked items from either individual list. Conversely, if k is increased significantly (e.g., up to 500), the scoring curve flattens out, minimizing the impact of top rank positions and allowing a document's presence across multiple lists to determine its final placement.

Q4: What are the semantic implications if a target text chunk is returned with a high rank in the dense retrieval list but is absent from the top-20 BM25 results?

****Answer:****

This performance pattern indicates that the document is highly relevant to the conceptual meaning of the user query but does not contain the exact vocabulary tokens or keyword strings used in the prompt. This scenario frequently occurs when queries use conversational phrasing, synonyms, or descriptive language instead of precise technical terms (e.g., a user asking "how to fix a leaking roof" matching a technical manual titled "Addressing Structural Moisture Ingress"). The dense encoder successfully identifies and matches these underlying semantic concepts, while the BM25 model fails to match the specific terms and ranks the document outside its top-20 results.

Q5: Why is the retrieval phase of this RAG architecture run locally on the CPU while text generation is dispatched to an external API service like Groq?

****Answer:****

This hybrid architecture optimizes performance by separating the resource demands of the retrieval and generation phases. The local retrieval phase uses a highly optimized, compact embedding model (`all-MiniLM-L6-v2`) combined with a FAISS index, which can execute dense matrix operations across a small corpus of 70 chunks in less than two milliseconds on a standard CPU. This eliminates unnecessary network latency, security exposure, and API usage costs during search operations. However, high-quality text generation requires a large language model with massive parameter scales (such as `llama-3.3-70b`). Running an LLM of this size locally requires specialized, expensive GPU infrastructure that is impractical for standard client workstations. Delegating this generation step to Groq's dedicated hardware platform ensures high-speed token generation over an encrypted connection without requiring local high-performance compute resources.

Q6: How do tokenization differences between your dense embedding model

and your sparse BM25 engine affect how they handle exact-term queries?

****Answer:****

The sparse BM25 engine uses a custom regular expression tokenization strategy designed to preserve whole alphanumeric strings, structural dash marks, and decimal points as single discrete units (e.g., keeping strings like `ABC-2024-117` or `8.2.1` completely intact). This allows the BM25 index to reliably perform exact keyword matches when these specific identifiers appear in user queries. In contrast, the dense vector engine uses a WordPiece or Byte-Pair Encoding (BPE) sub-word tokenizer. This approach frequently breaks unfamiliar technical terms or novel alphanumeric serial numbers down into generic sub-word components (e.g., splitting `ABC-2024-117` into `["abc", "-", "20", "24", "-", "117"]`). This sub-word tokenization flattens the unique feature profiles of technical terms, making it difficult for the dense embedding model to differentiate between similar but distinct serial codes.

Q7: What occurs mathematically within the RRF algorithm when a specific document chunk appears in only one of the two candidate retrieval lists?

****Answer:****

When a document chunk appears in only a single candidate list, it contributes a value of zero to the summation calculation for the missing list. For example, if a document ranks #5 in the dense retrieval list but fails to qualify for the top-20 BM25 list, its final RRF score calculation is computed purely from its single valid rank:

$$\text{Score}_{\text{RRF}} = \frac{1}{60 + 5} + 0 = \frac{1}{65} \approx 0.0153846$$

The document can still secure a position in the final top-5 merged list if its single-list score is high enough. However, it will naturally rank lower than competing documents that appear consistently across both retrieval lists, as those items benefit from combined reciprocal fractional scores from both search paths.

Q8: How does the system handle a scenario where the total character count of the top-5 retrieved context chunks exceeds the context limit of the LLM prompt?

****Answer:****

The system manages context limits using a defensive prompt budgeting strategy implemented in the core execution scripts. Each document chunk in the synthetic corpus is capped at a maximum length of 1,000 characters, ensuring that the combined length of the top-5 retrieved chunks never exceeds 5,000 characters. This strict local limit guarantees that the assembled prompt context fits easily within the 128k token context window of the `llama-3.3-70b-versatile` model on Groq. If a chunk ever exceeds this character limit during runtime, the context assembly system clips the text to the maximum character boundary, appends a truncation warning marker to the prompt, and writes an operational alert entry to `output.log` to document the change.

Q9: Explain the operational differences, trade-offs, and failure behaviors between the Fail-Fast and Graceful Degradation error handling strategies used in this project.

****Answer:****

The pipeline uses a combination of both error-handling strategies depending on the severity of the operational failure:

* **Fail-Fast Strategy**: This strategy is applied during initial system startup to validate critical configuration parameters. If crucial environmental variables (like `GROQ\_API\_KEY`) are missing, or if core data assets (like `corpus.json`) are absent from the disk, the application logs a critical exception and terminates immediately with a non-zero exit code. This prevents the system from running a broken pipeline or wasting API resources on unauthenticated requests.

* **Graceful Degradation Strategy**: This strategy is applied within the core evaluation loops. Individual evaluation queries are isolated within localized try-except blocks. If a single query fails due to an unexpected text formatting issue or retrieval anomaly, the system logs the error with the corresponding query identifier, updates an internal skipped item counter, and immediately continues to the next test item. This ensures that a minor error in a single data row cannot interrupt a large batch evaluation run.

Q10: How would this local hybrid retrieval architecture need to be adapted to scale efficiently to an enterprise corpus containing millions of documents?

Answer:

Scaling this hybrid architecture to support millions of documents requires transitioning from in-memory data structures to distributed enterprise search systems:

1. **Dense Retrieval Upgrade**: The in-memory FAISS flat index (`IndexFlatIP`) performs a complete brute-force search across all vector indices, which becomes too slow as the collection grows. This must be replaced with an Hierarchical Navigable Small World (HNSW) index or an Inverted File (IVF) index to enable fast, approximate nearest neighbor searches.
2. **Sparse Retrieval Upgrade**: The pure-Python `rank\_bm25` library stores its inverted text indices directly in system RAM, which will cause memory exhaustion errors with millions of documents. This needs to be replaced with a dedicated distributed search engine like Elasticsearch or OpenSearch, which can handle heavy keyword indexing and query workloads using optimized disk storage.
3. **Database Integration**: The simple JSON file storage model should be replaced with an enterprise database or a unified vector platform (such as Qdrant, Milvus, or pgvector) capable of orchestrating dense vector tracking, sparse keyword queries, metadata filtering, and RRF calculations within a single high-performance system.

The contents of this document are now complete and fully prepared to guide the advanced RAG implementation. Ensure all file names and paths match the design layout specifications during installation.